MANCHESTER 1824
The University of Manchester

MATH49111/69111: Scientific Computing

A first glance at C++ syntax

Matthias Heil

M.Heil@maths.manchester.ac.uk

A first glance at C++ syntax

- We're now going to write, compile, and run a short C++ program
- Relax: the code discussed should "make sense", but we'll revisit all the commands used properly later on!

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

This program evaluates and displays the result of the sum

$$\sum_{i=1}^{N} \frac{1}{i^2}$$

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- C++ source code consists of a sequence of *statements*, terminated with a semicolon, `;` , and executed in order
- *Compound statements* (*blocks*) are contained within braces `{ }`

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- The source is case-sensitive (`n_terms` $\neq$ `N_terms`)
- Indentation, whitespace and new lines are free-form (unlike Python). However, it's good to be consistent re layout (matching braces, amount of indentation, etc.)

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- *Comments* are notes written by the programmer that are ignored by the computer
- Any text after `//` at the end of a line, or between `/* and */`, is interpreted as a comment

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- `#include <iostream>` allows use of the `iostream` library, which defines functions to display text to the screen

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- This *function definition* defines the `main` function, which is called when the program starts.
- The code between braces `{ }` is the code associated with this function, called the function *body*.

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;          // Set up variables
  int n_terms = 100, i=1;    // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- These *declarations* create two new *variables* and *initialise* them. Variables must be declared before they can be used.
- The *type* of the variable `sum` is `double`, an approximation to a real number ($\mathbb{R}$). The type of `n_terms` and `i` is `int`, an integer

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- The `while` statement loops (executes repeatedly) the code in the following braces `{ }`, until a condition (`i`$\leq$`n_terms`) is false.
- Here we loop while `i`≤ 100

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- This *expression* is repeatedly executed by the `while` loop
- It increases the variable `sum` by the value `1.0/(i*i)`

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- This *expression* increases the variable `i` by 1.
- Since we start with `i`$= 1$ and increase `i` by 1 each time, the loop condition `i<=n_terms` becomes false after 100 loops.

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- This statement displays results of the calculation on screen.
- It uses the `std::cout` (standard output) and `std::endl` (end line) objects from the `<iostream>` library, included previously.

A first glance at C++ syntax

```
#include <iostream>

int main()
{
  double sum = 0.0;         // Set up variables
  int n_terms = 100, i=1;   // and initialise them
  while (i<=n_terms) /* Loop and sum terms */
  {
      sum += 1.0/(i*i);
      i++; // increase i by one
  }
  std::cout << "Sum of sequence after " << n_terms
            << " terms = " << sum << std::endl; // output result
  return 0;
}
```

- A `return` statement immediately exits a function.
- Returning from the `main` function exits the program.
- By convention, the `main` function returns 0 if the execution was successful, or any other integer if there was an error. The non-zero number returned is often used to indicate the type of error.

Let's compile and run it

Here's what happens if I compile and run this code (saved in a file called `cpp_syntax.cpp`) on my linux laptop:

```
> # Compile, using the g++ compiler; this creates
> # an executable called cpp_syntax:
> g++ -o cpp_syntax cpp_syntax.cpp
>
> # Run the code
> ./cpp_syntax

Sum of sequence after 100 terms = 1.63498
```

- Consult the document `compiling_in_cluster.pdf` to find out how to do this on the computers in the University's computer labs (or find out how to do this on your own computer).